

AGENTS

AGENTS.md — HelixLLM

NON-NEGOTIABLE PRIME DIRECTIVE: “We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This **MUST NOT** be the case and execution of tests and Challenges **MUST** guarantee the quality, the completion and full usability by end users of the product!” This statement is the foundational requirement of this project. Any agent dispatch, any code review that allows green tests on broken features is a violation and **MUST** be rejected.

This file defines agent constraints, capabilities, and collaboration rules for automated agents working on the HelixLLM codebase.

Project Overview

Submodule: HelixLLM

Repository: `github.com/HelixDevelopment/HelixLLM`

Mission: HelixLLM is the LLM abstraction layer providing a multi-provider fallback chain with local llama.cpp as guaranteed last-resort fallback. It serves OpenAI-compatible and Anthropic-compatible APIs.

This file assumes the reader knows nothing about the project. Deeper AGENTS.md or CLAUDE.md files in subdirectories take precedence over this root file for files within those subtrees.

Universal Mandatory Constraints (Inherited from Constitution)

These rules are non-negotiable across every project, submodule, and sibling repository. Each project **MUST** surface them in its own CLAUDE.md, AGENTS.md, and CONSTITUTION.md. Project-specific addenda cannot weaken or override these.

Hard Stops (permanent, non-negotiable)

1. **NO CI/CD pipelines.** No `.github/workflows/`, `.gitlab-ci.yml`, Jenkinsfile, `.travis.yml`, `.circleci/`, or any automated pipeline. No Git hooks either.
2. **NO HTTPS for Git.** SSH URLs only (`git@github.com:...`, `git@gitlab.com:...`) for clones, fetches, pushes, and submodule updates.
3. **NO manual container commands.** Container orchestration is owned by the project’s binary/orchestrator. Direct `docker/podman start|stop|rm` and `docker-compose up|down` are prohibited as workflows.

Mandatory Development Standards

1. **100% Test Coverage.** Every component **MUST** have unit, integration, E2E, automation, security/penetration, and benchmark tests.
2. **Challenge Coverage.** Every component **MUST** have Challenge scripts validating real-life use cases.
3. **Real Data.** Beyond unit tests, all components **MUST** use actual API calls, real databases, live services.
4. **Health & Observability.** Every service **MUST** expose health endpoints. Circuit breakers for all external dependencies.
5. **Documentation & Quality.** Update `CLAUDE.md`, `AGENTS.md` alongside code changes. Conventional Commits.
6. **Validation Before Release.** Pass the project's full validation suite plus all challenges.
7. **No Mocks or Stubs in Production.** **STRICTLY FORBIDDEN** in production code. Only unit tests may use mocks/stubs.
8. **Comprehensive Verification.** Runtime testing, compile verification, code structure checks. Grep-only validation is **NEVER** sufficient.
9. **Resource Limits for Tests & Challenges (CRITICAL).** 30-40% of host resources. Use `GOMAXPROCS=2, nice -n 19, ionice -c 3, -p 1`.
10. **Bugfix Documentation.** All bug fixes **MUST** be documented in `docs/issues/fixed/BUGFIXES.md`.
11. **Real Infrastructure for All Non-Unit Tests.** Mocks/fakes **MAY** be used **ONLY** in unit tests. **ALL** other test types **MUST** execute against the **REAL** running system.
12. **Reproduction-Before-Fix (CONST-032 — MANDATORY).** Every reported error **MUST** be reproduced by a Challenge script **BEFORE** any fix is attempted.
13. **Concurrent-Safe containers (CONST-029).** Mutable shared collections **MUST** use `safe.Store[K,V]` / `safe.Slice[T]`. Bare `sync.Mutex` + `map/slice` is prohibited for new code.

Definition of Done (universal)

A change is **NOT** done because code compiles and tests pass. “Done” requires pasted terminal output from a real run, produced in the same session as the change.

- **No self-certification.** Words like *verified*, *tested*, *working*, *complete*, *fixed*, *passing* are forbidden unless accompanied by pasted output.
 - **Demo before code.** Every task begins by writing the runnable acceptance demo.
 - **Real system, every time.** Demos run against real artifacts.
 - **Skips are loud.** `t.Skip` / `@Ignore` / `xit` without `SKIP-OK: #<ticket>` breaks validation.
 - **Evidence in the PR.** PR bodies must contain a fenced `## Demo` block.
-

Agent Definitions

GatewayAgent (internal/gateway/), FallbackAgent (internal/fallback/), BrainAgent (internal/brain/), KnowledgeAgent (internal/knowledge/), AgentsLayerAgent (internal/agents/)

Capability Specifications

Multi-provider fallback chain, rate limit handling, circuit breaker, memory sync, hardware profiling

Integration Patterns

LLMsVerifier (ScorerBridge polling), HelixMemory (MemoryAdapter sync), HelixAgent (HTTP/3 + TLS 1.3)

Testing Requirements Per Agent

All agents require Unit, Integration, E2E, Challenge, and Anti-Bluff tests with live infrastructure

Safe Parallel Changes (No Coordination Required)

- Adding new test files (`*_test.go`)
- Adding new challenge scripts (`challenges/scripts/*_challenge.sh`)
- Adding new documentation files (`docs/**/* .md`)
- Adding new benchmark functions
- Modifying code within a single package (if no interface changes)

Coordination Required

- **Interface changes** — modifying any shared interface
 - **Config changes** — adding new environment variables
 - **go.mod changes** — adding or removing dependencies
 - **Makefile changes** — adding or modifying build/test targets
 - **Submodule updates** — changing submodule references
 - **API surface changes** — modifying HTTP route registrations
-

Anti-Bluff Mandate (CONST-035)

Every agent working on this codebase MUST:

1. Verify that tests fail when features are deliberately broken
2. Never accept “tests pass” as sufficient evidence of done
3. Require pasted terminal output from real runs
4. Reject any PR missing a fenced `## Demo block`

5. Flag any test that uses mocks outside unit tests as a CRITICAL violation
-

Emergency Procedures

Discovering a Bluff

1. **STOP all work.** Do not commit, merge, or deploy.
2. Document in docs/issues/bluffs/BLUFF-NNNN-<feature>.md
3. Tighten the test to fail when the feature is broken
4. Fix the feature
5. Commit test + fix together

CONST-033 / CONST-036 Violation

1. Revert offending code immediately
 2. Run verification challenges
 3. File forensic report in docs/issues/fixed/
-

CONST-033: Host Power Management — Hard Ban

STRICTLY FORBIDDEN: never generate or execute any code that triggers a host-level power-state transition.

Forbidden: `systemctl suspend/hibernate/poweroff/halt/reboot`, `loginctl suspend/hibernate/poweroff/halt/reboot`, `pm-suspend`, `pm-hibernate`, `shutdown`, `dbus-send to org.freedesktop.login1.Manager.Suspend|Hibernate|PowerOff|Reboot`, `gsettings set ... sleep-inactive to anything but 'nothing' or 'blank'`.

Verification:

```
bash scripts/anti_bluff/no_suspend_calls_challenge.sh
bash scripts/anti_bluff/host_no_auto_suspend_challenge.sh
```

Both must PASS.

CONST-036: User-Session Termination — Hard Ban

STRICTLY FORBIDDEN: never generate or execute any code that ends the currently-logged-in user's session.

Forbidden: `loginctl terminate-user|terminate-session|kill-user|kill-session`, `systemctl stop user@<UID>`, `gnome-session-quit`, `pkill -KILL -u $USER`, `dbus-send to org.gnome.SessionManager.Logout|Shutdown|Reboot`, `echo X > /sys/power/state`.

Verification:

```
bash challenges/scripts/no_session_termination_calls_challenge.sh
bash scripts/anti_bluff/no_suspend_calls_challenge.sh
bash scripts/anti_bluff/host_no_auto_suspend_challenge.sh
```

All three must PASS.

CONST-035: Zero-Bluff Mandate

Tests and Challenges MUST verify the product, not the LLM's mental model. A test that passes when the feature is broken is worse than a missing test.

- TCP-open is the FLOOR, not the ceiling.
- No mocks/fakes outside unit tests.
- Functional verification of every claim.
- Re-verify after every change.
- End-user usability mandate: every PASS certifies Quality, Completion, AND Full Usability.

Bluff taxonomy: Wrapper bluff, Contract bluff, Structural bluff, Comment bluff, Skip bluff.

CONST-037 through CONST-040: LLMsVerifier and QA Mandates

CONST-037: LLMsVerifier Verification Mandate

LLMsVerifier is the single source of truth. Verified models MUST carry (`llmsvd`) suffix.

CONST-038: CLI Agent Configuration Mandate

ALL CLI agent configs MUST be generated through `pkg/cliagents/`. 48 agents total.

CONST-039: Challenge System Integrity Mandate

Every component MUST have Challenge scripts. Challenge scripts MUST exit non-zero when broken.

CONST-040: No Self-Certification Mandate

Words like *verified*, *tested*, *working*, *complete*, *fixed*, *passing* are FORBIDDEN without pasted terminal output.

Submodule-Specific Notes

The Gateway NEVER calls Brain directly. Always dispatch through `gateway.Completer`. `FallbackChain` implements `Completer`; individual providers are wrapped by `brain.ProviderCompleter`.
